What is a comment?

- A comments is special syntax

Why comment?

- A comment is used for telling to compiler for ignoring some part of the code compilation

- The code placed in the comment will not be compiled by compiler, byte code is not generated

- It means the code placed in comment will be not be available in .class file

When must use a comment?

We must use comment

- When the existing code is temporarily not required or
- When we need to provide description to a program

How many types of comments Java supports?

- Java supports 3 types of comments
  1. Single line comment //
  2. Multi line comment /* */
  3. Document comment /** */

- Single line and multiline comments are used for commenting programming statements such as variables, methods, Sopln(), conditions, loops, etc...

- Document comment is used for providing description to programming elements such as class, variable, method, constructor, block, inner classes.

Below program shows the usage of above three comments

```
/**
 * This class is used for adding given numbers
 *
 */
class Addition {
```

```
//    int a = 10;
//    int b = 20;

/*
  void add(){
    int c = a + b;
    System.out.println(c);
  }
*/

/**
 *  This method is used for adding two given numbers
 *  and prints the result
 */
  void add(int a, int b) {
     int c = a + b;
     System.out.println(c);
  }
}
```

Identifier, Types of identifiers and rules

==========

1. A name of a programming element is technically called as an identifier

2. A class name, variable name, method name are called as identifier

3. For example:

```
    class Addition {

        int a = 10;

        void add() {

          System.out.println("add");
        }
    }
```

Addition, a, add, System, out, println --> Identifier name

4. If we create a class or variable or method
without name we will get CE: <identifier> expected

```
class {

        int  = 10;

        void (){ }

}
```

5. Rules in creating identifier
We have 7 rules on creating an identifier

1. Identifier must contain only
    1. Letters -> A-Z or a-z
    2. Digits -> 0-9
    3. Special characters -> only $ and _

2. Identifier cannot be started with a digit
We can use a digit only from second character onwards

3. Identifier cannot have special characters except $ and _

4. Identifier cannot have space in the middle of words
if we need to provide spaces, we must connect by
using _. Hence _ is called as connector symbol.

Note: It is a wrong naming convention using _ in names.
We must use _ only in final variable names

5. Identifier is case sensitive (a != A)
We can create two variables with the name 'a' and 'A'

6. The keyword and reserved word cannot be used as
an user defined identifier.

But predefined class name, variable name and method name
can used as user defined identifier.

Also there is not rule identifier length, we can
have any number of characters, but it not recommended,
because it is not readable.

7. From Java 9v onwards we cannot use single Underscore (_) as an identifier, because it is made as a keyword.

Note: also we cannot use single digit or a number as identifier. We must use it in combination with either letter or $ or _.

6. Types of identifiers

We will provide name to the below programming elements

1. module name
2. package name
3. class name
4. variable name
5. method name
6. label name
7. generic parameter name

7. Identify valid identifiers in the below list

Keywords, rules, list of keywords, Reserved words
As part of Java 21v we have 71 reserved words among them

1. 51 are keywords
2. 10 are restricted keywords
3. 7 are special identifiers
4. 3 are literals

Below are the list of 71 reserved words

==========

01. Package creation and usage (2)
    01. package
    02. import
02. Class creation (3)
    03. class
    04. interface
    05. enum (1.5)
03. Data types and Return types (8 + 1)
    06. byte
    07. short

08. int
09. long

10. float
11. double

12. char
13. boolean

14. void

04. Memory allocation (2)
static
15. new

05. Control flow statements (11)

01. Conditional or Decision Making (5)
16. if
17. else

18. switch
19. case
20. default

02. Loop or Iterative (3)
21. while
22. do
23. for

03. Transfer or Branching (3)
24. break
25. continue
26. return

06. Accessibility Modifiers (4)
27. private
<default>/<package-private>
28. protected
29. public

07. Execution level Modifiers (8)
30. static 34. transient
31. final 35. volatile
32. abstract 36. synchronized
33. native 37. strictfp (1.2v) (17v*)

08. Establishing inheritance relationship (2)

38. extends
39. implements

09. Representing object and its members (3)
40. this
41. super
42. instanceof

10. Exception Handling statements (5 + 1)
43. throw
44. throws

45. try
46. catch
47. finally

48. assert (1.4v)

11. Unimplemented keywords (2)
49. goto
50. const

12. Java 9v new keyword (1)
51. _ (Underscore)

13. Default/Reserved literals (3)
- boolean literals(2)
52. true
53. false

- Null literal (1)
54. null

14. Java 9v Restricted keywords (10)
55. module

56. requires
57. transitive

58. exports
59. open
60. opens
61. to

62. provides
63. with

64. uses

15. Special Identifiers or Restricted Identifiers (6)

65. var (Java 10)

	Preview	Permanent
66. yield	(Java 13 ->----->	Java 14)
67. record	(Java 14 -> 15 ->	Java 16)
68. sealed	(Java 15 -> 16 ->	Java 17)
69. non-sealed	(Java 15 -> 16 ->	Java 17)
70. permits	(Java 15 -> 16 ->	Java 17)
71. when	(Java 19 -> 20 ->	Java 21)